

DAM CFGS Desenvolupament d'Aplicacions Multiplataforma
Mòdul 0486 – Accés a dades
U1 – Persistència en fitxers
EAC1
(Curs 2025–26 / 2n semestre)

Presentació i resultats d'aprenentatge

Aquest exercici d'avaluació continuada (EAC) es correspon amb els continguts treballats a la unitat formativa **1 Persistència en fitxers**.

Els **resultats d'aprenentatge** que es plantegen són:

1. Desenvolupa aplicacions que gestionen informació emmagatzemada en fitxers identificant-ne el camp d'aplicació i utilitzant classes específiques.
 - 1.1 Utilitza classes per a la gestió de fitxers i directoris.
 - 1.2 Valora els avantatges i els inconvenients de les diferents formes d'accés.
 - 1.3 Utilitza classes per recuperar informació emmagatzemada en fitxers.
 - 1.4 Utilitza classes per emmagatzemar informació en fitxers.
 - 1.5 Utilitza classes per fer conversions entre diferents formats de fitxers.
 - 1.6 Preveu i gestiona les excepcions.
 - 1.7 Prova i documenta les aplicacions desenvolupades.

Criteris d'avaluació

La puntuació màxima assignada a cada activitat s'indica **al final del document**.

Els criteris que es tindran en compte per avaluar el treball de l'alumnat són els següents:

- Utilització de les classes per a la gestió de fitxers i directoris.
- Utilització de les diferents formes d'accés.
- Utilització de les classes per emmagatzemar/recuperar informació en un fitxer.
- Ús de les excepcions.
- Documentació del codi.
- Funcionament correcte del programa.

Forma i data de lliurament

Un cop finalitzat l'exercici d'avaluació continuada heu d'enviar el document des de l'apartat **Lliurament EAC1** de l'aula, dins del termini establert. Tingueu en compte que el sistema no permetrà fer lliuraments després de la data i hora indicades.

El nom del fitxer serà el següent: **DAM_0486B0_EAC1_Cognom1_Inicial del cognom2**. Els cognoms s'escriuran sense accents. Per exemple, l'estudiant *Joan García Santos* posaria el següent nom al seu fitxer de l'EAC1: **DAM_0486B0_EAC1_Garcia_S**.

El termini de lliurament finalitza a les **23:55 h** del dia **17/03/2026**. La proposta de solució i les qualificacions de l'EAC es publicaran el dia 24/03/2026.

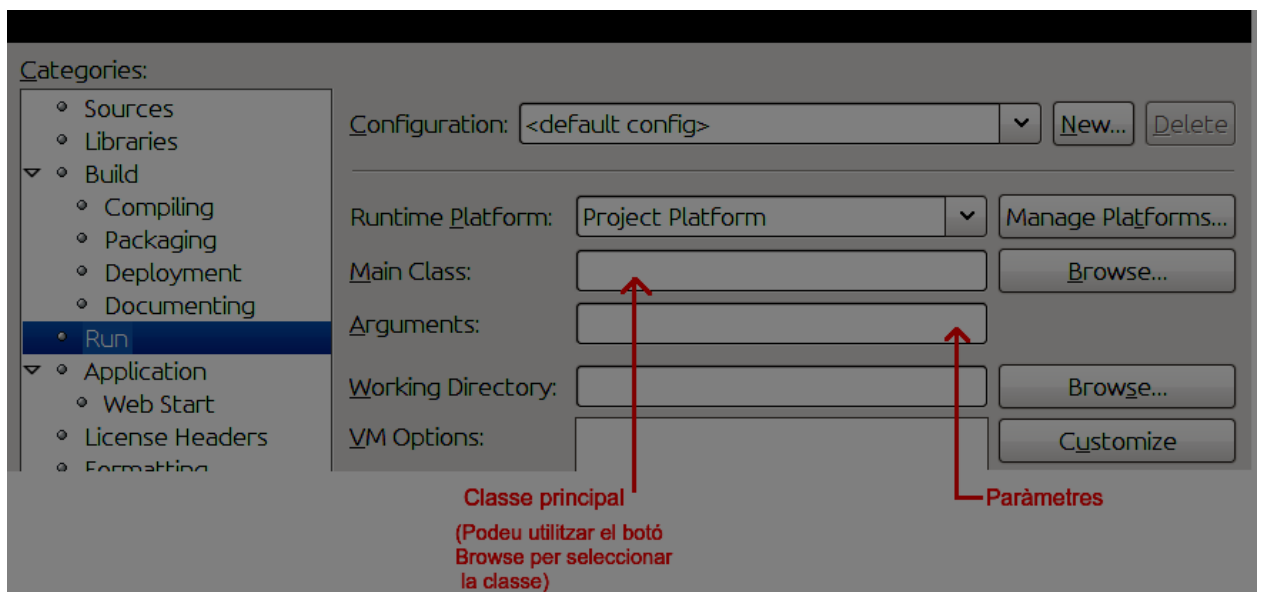
	Codi: I71	Exercici d'avaluació contínua 1	Pàgina 1 de 17
	Versió: 03	DAM_0486B0_EAC1_Enunciat_2526S2	Lliurament: 17/03/2026

Enunciat

Aquest enunciat va adjunt amb un fitxer comprimit que conté la carpeta *src* i el fitxer *pom.xml* del projecte que heu de completar, un fitxer *.xml* i un fitxer *.json*. Per a publicar la solució heu d'entregar un fitxer que serà un comprimit **.zip**, que s'anomenarà com s'ha indicat a l'apartat anterior i que contindrà **aquesta carpeta src** d'aquest projecte, un cop l'hàgiu completat a fi de resoldre els exercicis demanats. Heu de posar totes les solucions en un únic projecte. Al fitxer comprimit que lliurareu **no cal incloure aquest document** que esteu llegint.

Al primer exercici cal crear una classe amb un mètode estàtic *main* (programa principal) que rep paràmetres en la línia d'ordres. Aquests paràmetres es poden indicar d'una d'aquestes dues maneres:

- Si es fa la crida a la classe directament des de la **línia d'ordres del sistema**, afegint-los després del nom de la classe (la crida es faria escrivint `java <nom de la classe> <paràmetres>`).
- Si ho feu des de l'entorn **NetBeans**, cal que a les propietats del projecte, apartat "Run", seleccioneu la classe principal i escriviu els paràmetres del programa principal, tal com s'indica en la imatge que segueix. Un cop fet això, ja podeu executar el programa normalment.

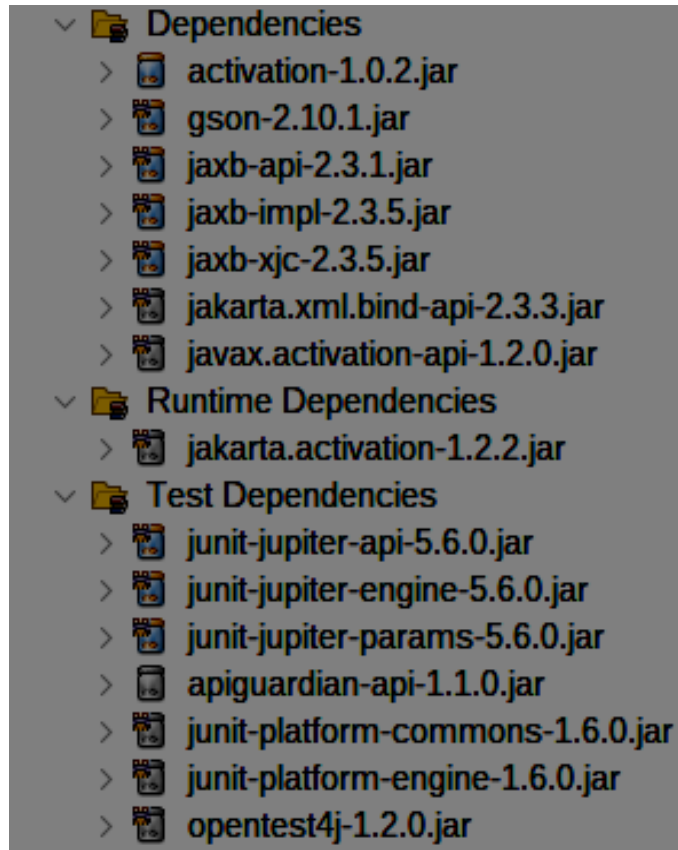


IMPORTANT

- No es pot utilitzar cap biblioteca (*library*), a part de la pròpia del *JDK*, versió 8 o superior del llenguatge, la de *JAXB* si es fa servir una versió superior a 8, i la de *Gson*.
- El tractament dels errors s'ha de fer, sempre que sigui possible, utilitzant excepcions.
- El **codi ha d'estar suficientment documentat** (és l'única documentació que es demana). Ben documentat no vol dir "sobre-documentat". Només cal escriure comentaris a les línies on convingui un aclariment per entendre millor el codi, a més d'omplir el necessari per generar els fitxers *javadoc* (*NetBeans* us pot ajudar a generar aquests comentaris necessaris pel *javadoc*). Una màxima a la programació diu que el codi millor comentat és el que no necessita cap comentari perquè s'ha escrit d'una manera prou entenedora. Això quasi sempre és una utopia, però és cap on cal acostar-s'hi.

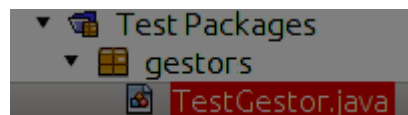
	Codi: I71	Exercici d'avaluació contínua 1	Pàgina 2 de 17
	Versió: 03	DAM_0486B0_EAC1_Enunciat_2526S2	Lliurament: 17/03/2026

En el projecte que trobareu adjunt a aquest enunciat hi ha un fitxer pom.xml que conté totes les biblioteques necessàries per a usar JUnit5, JAXB i Gson. Quan obriu el projecte amb Netbeans, trobareu les biblioteques de JAXB i Gson a la carpeta *Dependencies* en l'arbre del projecte, i les biblioteques de JUnit5 a la carpeta *TestDependencies*.



A l'exercici 2 s'inclou, a la carpeta *test* del comprimit, una classe, *TestGestor* que cal utilitzar per comprovar el funcionament correcte de la biblioteca tot utilitzant *JUnit5*. A l'entorn *NetBeans* apareixerà dins de *TestPackages*.

Per a fer la prova de la biblioteca amb *JUnit*, només heu de seleccionar l'opció *Test File* del menú contextual que es mostra en posar el ratolí a sobre de la classe *TestGestor*, que apareix dins del paquet *gestors*, a *TestPackages*. També podeu utilitzar aquesta classe per a depurar el vostre programa seleccionant l'opció *Debug Test File* (en lloc de l'opció *Test File*). A continuació teniu una imatge que mostra com s'ha de veure la classe abans d'activar el menú contextual:



	Codi: I71	Exercici d'avaluació contínua 1	Pàgina 3 de 17
	Versió: 03	DAM_0486B0_EAC1_Enunciat_2526S2	Lliurament: 17/03/2026

EXERCICI 1 (5 punts)

En el fitxer comprimit hi trobareu el paquet *ex1*, que inclou la classe corresponent a aquest primer exercici: **Eac1**, que conté el mètode estàtic *main* (és a dir, el programa principal).

Aquest mètode *main*:

- Haurà de copiar o moure fitxers del directori de treball a la carpeta indicada.
- Rebrà 4 o 5 **paràmetres**, per aquest **ordre**:
 - **Primer** paràmetre: serà la lletra '**C**' o la lletra '**M**' (no distingeix majúscules de minúscules), amb el següent significat, segons el cas:
 - **C** (Copiar): Indica l'operació de copiar els fitxers (no directoris) seleccionats
 - **M** (Moure): Indica l'operació de moure els fitxers (no directoris) seleccionats. Si la carpeta origen indicada en el primer paràmetre no té permís d'escriptura, cal mostrar un missatge indicant que l'operació no es pot fer.
 - **Segon** paràmetre: nom de la carpeta on es copiaran o mouran els fitxers seleccionats, la qual ha de tenir permís d'escriptura, i en cas que no en tingui cal mostrar un missatge.
 - **Tercer** paràmetre: serà la lletra '**M**' o '**N**' (no distingeix majúscules de minúscules), i indicarà com es seleccionen els fitxers, amb el següent significat:
 - **M** (Mida): Anirà seguit d'un **quart** paràmetre que indicarà la mida màxima en bytes dels fitxers que es seleccionaran per a copiar o moure.
 - **N** (Nom): Anirà seguit d'un **quart** paràmetre cadena de caràcters que serà l'inici del nom dels fitxers (sense incloure carpeta) que es seleccionaran per a copiar o moure.
 - **Cinquè** paràmetre opcional:
 - **H** (Hidden): No distingeix majúscules de minúscules. Indicarà que també es seleccionaran els fitxers ocults per copiar o moure

Com a **control d'errors**, escriurà un **missatge** a l'objecte *System.err* i acabarà el programa amb codi d'error **2** en qualsevol d'aquests casos:

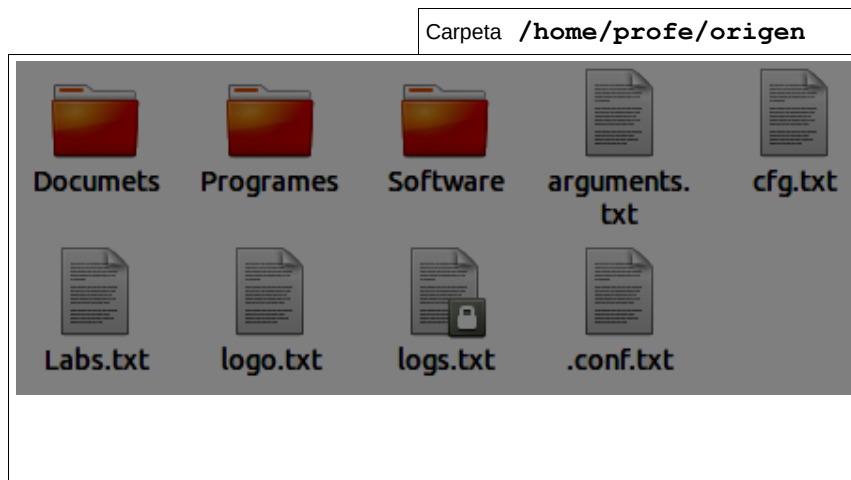
- El nombre de paràmetres no és **entre quatre i cinc**.
- El **primer** paràmetre no és ni una 'C' ni una 'M'
- El **segon** paràmetre no és una **carpeta** vàlida o no s'hi té permís d'escriptura
- El **tercer** paràmetre no és ni una 'M' ni una 'N'
- El **quart** paràmetre ha de ser un número enter, si el quart paràmetre és una 'M'
- El **cinquè** paràmetre no és una 'H'

Per trobar els fitxers que cal copiar o moure, **és obligatori fer una crida al mètode *listFiles*** de la classe *File*, tot passant-li un objecte de la classe ***FileFilter*** com a paràmetre. No es considerarà correcte cridar a *listFiles* sense paràmetres i fer el filtratge a la vegada que es copien o mouen els fitxers.

- **Per cada fitxer** copiat o mogut mostrarà el nom, acabat amb "(H)" si és un arxiu ocult, (per tant, el fitxer ocult git seria "git (H)") i la mida en Bytes acabat amb B.
- Per cada fitxer que no tingui permís de lectura, mostrar un missatge
- Al final farà un resum indicant el nombre de fitxers moguts o copiats i la mida total.

Tot seguit, teniu un exemple amb una possible carpeta i, a continuació, diferents exemples d'execucions d'aquest mètode *main* de la classe *Exercici1*.

	Codi: I71	Exercici d'avaluació contínua 1	Pàgina 4 de 17
	Versió: 03	DAM_0486B0_EAC1_Enunciat_2526S2	Lliurament: 17/03/2026



```

Command Prompt
root@PC:~$ Exercici1 c /home/profe/desti m 1000 h

logs.txt 2290 B
cfg.txt 2899 B
arguments.txt 1234 B
.conf.txt (H) 1254 B

4 fitxers copiats, 7677 B

root@PC:~$

```

Exemple 1: execució del programa des de línia d'ordres amb opció *c* (copiar) i *m* (indicant la mida)

```

Command Prompt
root@PC:~$ Exercici1 m /home/profe/desti n log

logo.txt 100 B
logs.txt 140 B

2 fitxers moguts, 240 B

root@PC:~$

```

Exemple 2: execució del programa des de línia d'ordres amb opció *e* i indicant les primeres lletres del nom

A la primera figura es pot veure el contingut d'un directori, *origen*, d'un sistema de fitxers (primer paràmetre). Cal **observar** que el fitxer *.conf.txt* és **ocult**. A les següents, com es fa la crida al mètode *main* de la classe *Exercici1* amb els paràmetres corresponents. Al primer cas, el resultat és la còpia dels fitxers amb els paràmetres proporcionats i en el segon, el canvi de lloc (o de nom) dels fitxers que contenen la cadena 'log' en el seu nom.

Ajut (seguir-lo és opcional): com a la condició de filtratge hi intervenen uns quants factors, podeu, en primer lloc, crear un objecte d'una subclasse anònima de la classe *FileFilter*, assignar-lo a una variable *i*, a continuació, cridar al mètode *listFiles* passant-li aquesta variable. Aquest objecte es pot crear així:

Fent servir les **expressions lambda** (característica inclosa a Java a partir de la versió 8):

	Codi: I71	Exercici d'avaluació contínua 1	Pàgina 5 de 17
	Versió: 03	DAM_0486B0_EAC1_Enunciat_2526S2	Lliurament: 17/03/2026

```
FileFilter filtre = (File fitxer1) → { ... sobrecàrrega del mètode accept ...};
```

...i la crida:

```
File[] llistatItems = carpetaDesti.listFiles(filtre);
```

PUNTUACIÓ

Els cinc punts de l'exercici estaran repartits de la següent manera:

Comentaris al codi0,5 punts
Gestió dels paràmetres i control d'errors.....1 punt
Ús de *FileFilter*.....2,5 punts
Mostrar el resultat.....1 punt

	Codi: I71	Exercici d'avaluació contínua 1	Pàgina 6 de 17
	Versió: 03	DAM_0486B0_EAC1_Enunciat_2526S2	Lliurament: 17/03/2026

EXERCICI 2 (5 punts)

Modifiqueu i completeu la biblioteca compresa en el paquet `ex2` (a la carpeta `src`) del fitxer comprimit adjunt. Aquesta biblioteca realitza una gestió senzilla d'una estació d'esquí, tot representat amb JSON i XML.

Aquesta biblioteca ha de permetre llegir i escriure dades de l'estació i les seves pistes.

A més, al paquet `gestors` de la carpeta `test` trobareu la classe `TestGestor`. Aquesta classe permet comprovar el funcionament correcte de les altres classes amb **JUnit**.

Haureu de modificar les classes `Estacio` i `Pista` del paquet `ex2.model` per afegir les anotacions amb JAXB i completar-hi el mètode de la classe `Estacio` **actualitzaEstatObertura** i els mètodes de la classe `GestorEstacio`:

- **llegirFitxerXML**
- **llegirFitxerJSON**
- **gravarFitxerXML**
- **gravarFitxerJSON**

Als comentaris (estil javadoc) dels fitxers `.java` de les classes s'explica què ha de fer cada mètode. A continuació també detallem el funcionament d'aquests mètodes.

En tots els mètodes de la classe `GestorEstacio` heu de passar per paràmetre el nom del fitxer XML o JSON des d'on es llegirà o gravarà l'objecte.

Els mètodes `llegirFitxerXML` i `llegirFitxerJSON` retornaran un objecte de la classe `Estacio` obtenint les dades dels fitxers XML (amb **Unmarshaller**) i JSON (amb **fromJson**) respectivament.

En els mètodes `gravarFitxerXML` i `gravarFitxerJSON`, a més del nom del fitxer, heu de passar per paràmetre l'objecte de classe `Estacio` que cal gravar en el fitxer, en format XML (amb **Marshaller**) i JSON (amb **toJson**) respectivament.

El mètode **actualitzaEstatObertura** calcularà el percentatge de pistes obertes i actualitzarà l'atribut `estatObertura` de la classe `Estacio`.

Per a facilitar la compleció dels mètodes, aquests es troben marcats amb **//TODO** per a indicar on s'ha d'escriure codi. Si seleccioneu l'opció de menú *Windows* → *Action Items* us apareixerà una finestra amb tots els **//TODO** que falten per completar als projectes oberts.

Indicacions:

1. Trobareu les classes **Estacio** i **Pista** al paquet `ex2.model` del fitxer comprimit adjunt. Caldrà completar el mètode **actualitzaEstatObertura** de la classe **Estacio**.

2. Els fitxers **estacio.xml** i **estacio.json** tindran el mateix format que aquests (són els que cal utilitzar per a fer les proves):

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<estacio id="ST-001">
  <nom>Neu Gran</nom>
  <comarca>Cerdanya</comarca>
```

	Codi: I71	Exercici d'avaluació contínua 1	Pàgina 7 de 17
	Versió: 03	DAM_0486B0_EAC1_Enunciat_2526S2	Lliurament: 17/03/2026

```

<web>https://www.neugran.cat</web>
<altitud-maxima>2535</altitud-maxima>
<qualificacio>Molt bona</qualificacio>
<estat-obertura-percentatge>80</estat-obertura-percentatge>

<pistes>
  <pista id="A101">
    <nom>La Canal</nom>
    <longitud>1500</longitud>
    <desnivell>450</desnivell>
    <color>Negra</color>
    <oberta>true</oberta>
    <gruix-neu>60</gruix-neu>
    <qualitat-neu>Pols</qualitat-neu>
  </pista>
  <pista id="A102">
    <nom>Pista del Bosc</nom>
    <longitud>2100</longitud>
    <desnivell>200</desnivell>
    <color>Blava</color>
    <oberta>true</oberta>
    <gruix-neu>45</gruix-neu>
    <qualitat-neu>Pols-Dura</qualitat-neu>
  </pista>
  <pista id="A103">
    <nom>Debutants I</nom>
    <longitud>500</longitud>
    <desnivell>50</desnivell>
    <color>Verda</color>
    <oberta>true</oberta>
    <gruix-neu>30</gruix-neu>
    <qualitat-neu>Dura</qualitat-neu>
  </pista>
  <pista id="A104">
    <nom>Muro de les Àligues</nom>
    <longitud>1800</longitud>
    <desnivell>600</desnivell>
    <color>Negra</color>
    <oberta>false</oberta>

```



```

    <gruix-neu>0</gruix-neu>
    <qualitat-neu>Sense neu</qualitat-neu>
  </pista>
  <pista id="A105">
    <nom>Camí Ral</nom>
    <longitud>3200</longitud>
    <desnivell>150</desnivell>
    <color>Verda</color>
    <oberta>true</oberta>
    <gruix-neu>40</gruix-neu>
    <qualitat-neu>Primavera</qualitat-neu>
  </pista>
  <pista id="A106">
    <nom>La Directa</nom>
    <longitud>1200</longitud>
    <desnivell>380</desnivell>
    <color>Vermella</color>
    <oberta>true</oberta>
    <gruix-neu>55</gruix-neu>
    <qualitat-neu>Pols</qualitat-neu>
  </pista>
  <pista id="A107">
    <nom>Costa dels Avets</nom>
    <longitud>2400</longitud>
    <desnivell>310</desnivell>
    <color>Vermella</color>
    <oberta>false</oberta>
    <gruix-neu>10</gruix-neu>
    <qualitat-neu>Dura</qualitat-neu>
  </pista>
  <pista id="A108">
    <nom>Pala Central</nom>
    <longitud>900</longitud>
    <desnivell>250</desnivell>
    <color>Vermella</color>
    <oberta>true</oberta>
    <gruix-neu>50</gruix-neu>
    <qualitat-neu>Pols-Dura</qualitat-neu>
  </pista>

```

```

<pista id="A109">
  <nom>Passeig de la Neu</nom>
  <longitud>4100</longitud>
  <desnivell>120</desnivell>
  <color>Blava</color>
  <oberta>true</oberta>
  <gruix-neu>35</gruix-neu>
  <qualitat-neu>Primavera</qualitat-neu>
</pista>
<pista id="A110">
  <nom>Tub de la Coma</nom>
  <longitud>1350</longitud>
  <desnivell>410</desnivell>
  <color>Negra</color>
  <oberta>true</oberta>
  <gruix-neu>70</gruix-neu>
  <qualitat-neu>Pols</qualitat-neu>
</pista>
</pistes>
</estacio>

```

```

{
  "id": "ST-001",
  "nom": "Neu Gran",
  "comarca": "Cerdanya",
  "altitud_maxima": 2535,
  "web": "https://www.neugran.cat",
  "qualificacio": "Molt bona",
  "estat_obertura_percentatge": 80,
  "pistes": [
    {
      "id": "A101",
      "nom": "La Canal",
      "longitud": 1500,
      "desnivell": 450,
      "color": "Negra",
      "oberta": true,
      "gruix_neu": 60,
      "qualitat_neu": "Pols"
    }
  ]
}

```

```

},
{
  "id": "A102",
  "nom": "Pista del Bosc",
  "longitud": 2100,
  "desnivell": 200,
  "color": "Blava",
  "oberta": true,
  "gruix_neu": 45,
  "qualitat_neu": "Pols-Dura"
},
{
  "id": "A103",
  "nom": "Debutants I",
  "longitud": 500,
  "desnivell": 50,
  "color": "Verda",
  "oberta": true,
  "gruix_neu": 30,
  "qualitat_neu": "Dura"
},
{
  "id": "A104",
  "nom": "Muro de les Àligues",
  "longitud": 1800,
  "desnivell": 600,
  "color": "Negra",
  "oberta": false,
  "gruix_neu": 0,
  "qualitat_neu": "Sense neu"
},
{
  "id": "A105",
  "nom": "Camí Ral",
  "longitud": 3200,
  "desnivell": 150,
  "color": "Verda",
  "oberta": true,
  "gruix_neu": 40,

```

```

    "qualitat_neu": "Primavera"
  },
  {
    "id": "A106",
    "nom": "La Directa",
    "longitud": 1200,
    "desnivell": 380,
    "color": "Vermella",
    "oberta": true,
    "gruix_neu": 55,
    "qualitat_neu": "Pols"
  },
  {
    "id": "A107",
    "nom": "Costa dels Avets",
    "longitud": 2400,
    "desnivell": 310,
    "color": "Vermella",
    "oberta": false,
    "gruix_neu": 10,
    "qualitat_neu": "Dura"
  },
  {
    "id": "A108",
    "nom": "Pala Central",
    "longitud": 900,
    "desnivell": 250,
    "color": "Vermella",
    "oberta": true,
    "gruix_neu": 50,
    "qualitat_neu": "Pols-Dura"
  },
  {
    "id": "A109",
    "nom": "Passeig de la Neu",
    "longitud": 4100,
    "desnivell": 120,
    "color": "Blava",
    "oberta": true,

```

```

        "gruix_neu": 35,
        "qualitat_neu": "Primavera"
    },
    {
        "id": "A110",
        "nom": "Tub de la Coma",
        "longitud": 1350,
        "desnivell": 410,
        "color": "Negra",
        "oberta": true,
        "gruix_neu": 70,
        "qualitat_neu": "Pols"
    }
]
}

```

Els fitxers que es gravaran amb el Test tindran de nom **estacio2.xml** i **estacio2.json**.

3. Per a poder llegir i escriure amb format XML primer que cal fer és posar anotacions a les classes **Estacio** i **Pista** per especificar quin serà el format dels seus objectes representats en XML. Concretament, caldrà utilitzar les que s'indiquen a l'annex. Veureu que no cal utilitzar-les totes. Com a mínim, hauríeu d'utilitzar `@XmlRootElement`, `@XmlType`, i `@XmlElementWrapper` en combinació amb `@XmlElement`. Penseu que els noms de les variables poden no ser sempre exactament iguals als de l'XML.

4. Per a poder llegir i escriure amb format Gson, només caldrà afegir anotacions `@SerializedName` per a relacionar el nom dels atributs de Java amb els noms del fitxer JSON, en cas que siguin diferents.

5. Un cop posades les anotacions, podeu afegir a les classes els comentaris necessaris per generar els *javadoc*.

6. Tot seguit, cal ja completar els mètodes de llegir i gravar XML, tenint en compte:

- Tant per llegir com per gravar en XML, crear un context de JAXB a partir de la classe els objectes de la qual seran els elements arrel (al nostre cas, *Estacio*). Això es fa cridant a `JAXBContext.newInstance(Estacio.class)`. Aquesta crida retorna un objecte del tipus `JAXBContext`.
- Per a llegir l'arxiu d'entrada xml, cal crear un objecte que implementi la interfície *Unmarshaller*; caldrà utilitzar aquest objecte per a convertir l'entrada XML llegida del respectiu fitxer en un objecte Java. L'objecte *Unmarshaller* es crea cridant sense paràmetres al mètode `createUnmarshaller` del context creat al primer punt. Un cop tenim aquest objecte *Unmarshaller*, el seu mètode `unmarshall` ens permet llegir el fitxer. El mètode `unmarshall` rep el fitxer com a paràmetre.
- Per a gravar un objecte Java en un fitxer XML, cal crear un objecte que implementi la interfície *Marshaller*. Aquest objecte es crea cridant sense paràmetres al mètode `createMarshaller` del context creat al primer punt. Un cop tenim aquest objecte *Marshaller*, el seu mètode `marshall` ens permet gravar un objecte en un fitxer en format XML. El mètode `marshall` necessita, com a paràmetres, l'objecte a gravar i el fitxer. Abans, però, de gravar l'objecte, és molt convenient fer la crida `setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true)`. `setProperty` és un mètode del mateix objecte de la classe *Marshaller* que hem esmentat. Aquesta crida aconsegueix que el fitxer XML quedi més llegible.

	Codi: I71	Exercici d'avaluació contínua 1	Pàgina 13 de 17
	Versió: 03	DAM_0486B0_EAC1_Enunciat_2526S2	Lliurament: 17/03/2026

7. Finalment, cal completar els mètodes de llegir i gravar JSON, tenint en compte:

- Tant per llegir com per escriure utilitzant la biblioteca Gson cal crear un objecte de la classe *Gson*.
- Per a llegir un fitxer JSON, podeu cridar el mètode *fromJson* passant com a primer paràmetre un objecte de la classe *FileReader* i com a segon paràmetre la classe de l'objecte que es vol llegir, en el nostre cas *Estacio.class*.
- Per a escriure en un fitxer JSON, cal utilitzar el mètode *toJson* passant com a paràmetre l'objecte que volem convertir, i obtindrem com a valor de retorn un String que caldrà escriure en el fitxer de sortida. Si volem que el fitxer resultant quedi més llegible podem utilitzar el mètode *setPrettyPrinting*.

En aquest enllaç podeu trobar un exemple senzill de JAXB que fa aquests passos:
<http://www.mkyong.com/java/jaxb-hello-world-example/>

Els casos de prova assumiran que el fitxers *estacio.xml* i *estacio.json* estan a l'arrel del projecte (és a dir, just a la carpeta del projecte de Netbeans) i són idèntics a l'esmentat en aquest enunciat.

PUNTUACIÓ

Els cinc punts de l'exercici estaran repartits de la següent manera

Anotacions 2 punts
Passar els casos de prova 3 punts (0,6 per test)

	Codi: I71	Exercici d'avaluació contínua 1	Pàgina 14 de 17
	Versió: 03	DAM_0486B0_EAC1_Enunciat_2526S2	Lliurament: 17/03/2026

ANNEX – Anotacions importants de JAXB

@XmlRootElement - Defineix l'element arrel del document XML. Admet l'atribut *name*, que indica el nom que tindrà l'element XML dins del qual s'insereix la seva representació.

Exemples:

- a) `@XmlRootElement` // anirà dins dels elements `<card>...</card>`
`public class Card {.....}`
- b) `@XmlRootElement(name="targeta")` // anirà dins dels elements `<targeta>...</targeta>`
`public class Card {.....}`

@XmlType(propOrder = {"element1",...}) - L'utilitzarem per indicar l'ordre dels elements fills dins la jerarquia. Per defecte, l'ordre depèn de la implementació.

Exemples:

```
@XmlRootElement(name="targeta")
@XmlType(propOrder = {"numero","titular","pin"})
public class Card {
    private int numero, pin;
    private String titular;

    .....
    // setters i getters d'aquestes tres dades
}
```

// l'estructura del fitxer XML serà:

`<targeta> <numero>...</numero><titular>...</titular><pin>...</pin></targeta>`

@XmlAccessorType – Defineix què vincula JAXB per defecte amb elements XML generats. Dit d'una altra manera, defineix què té en compte a l'hora de generar elements XML.

Hi ha quatre possibilitats:

@XmlAccessorType(Xml.AccessType.PUBLIC_MEMBER) – genera elements dades públiques, les dades amb anotacions i totes les propietats. És l'opció per defecte.

@XmlAccessorType(Xml.AccessType.FIELD) – vincula totes les dades i les propietats amb anotacions.

@XmlAccessorType(Xml.AccessType.PROPERTY) – vincula les dades amb anotacions i totes les propietats.

@XmlAccessorType(Xml.AccessType.NONE) – vincula les dades amb anotacions i les propietats amb anotacions.

Quan es parla de dades o propietats amb anotacions es refereix a anotades amb @XmlElement

Les propietats s'entenen implementades per setters i/o getters públics.

Exemple: tenim aquesta classe:

```
public class Card {  
    private int numero, pin;  
    public String titular;  
    .....  
    // setter i getter de la dada pin  
}
```

@XmlAccessorType(Xml.AccessType.PUBLIC_MEMBER) – generaria elements per titular (dada pública) i pin (propietat)

@XmlAccessorType(Xml.AccessType.FIELD) – generaria elements per numero, pin i titular, ja que tots són dades (encara que siguin privades).

@XmlAccessorType(Xml.AccessType.PROPERTY) – generaria només un element per la dada pin (propietat).

@XmlAccessorType(Xml.AccessType.NONE) – no generaria cap element

A més, si poséssim una anotació adient a alguna dada o propietat, a tots els casos anteriors també es generaria un element XML per aquesta dada o propietat.

	Codi: I71	Exercici d'avaluació contínua 1	Pàgina 16 de 17
	Versió: 03	DAM_0486B0_EAC1_Enunciat_2526S2	Lliurament: 17/03/2026

@XmlElement – Indica que cal vincular un element de Java amb un element XML. Aquest element pot ser una dada o una propietat (s'implementen amb setters i/o getters). Si no es posa aquesta anotació, l'element de Java serà mapat o no en funció del següent:

- Si no s'ha posat l'anotació **@XmlAccessorType**, es generen elements XML per les dades públiques i les propietats.
- Si s'ha posat l'anotació **@XmlAccessorType**, es generen elements XML en funció de la configuració d'aquesta anotació.

Admet l'atribut *name*, que indica el nom que tindrà l'element XML dins del qual s'inserirà la seva representació.

Exemples:

- a) **@XmlElement** // generarà l'element XML anomenat nif
`public int getNif()`
- b) **@XmlElement(name="identificador-fiscal")** // generarà l'element XML anomenat identificador-fiscal
`private String nif;`
- c) `public int getNif()` // es generarà l'element nif o no en funció de la configuració de l'anotació **@XmlAccessorType**

@XmlElementWrapper – Fa que els elements d'una col·lecció de Java es representin dins d'un element XML que els contingui. Admet l'atribut *name*, que indica el nom d'aquest element contenidor. Sol utilitzar-se en combinació amb **@XmlElement(name="...")** per indicar el nom que han de tenir els elements continguts.

Exemple:

```
@XmlRootElement(name="catalog")
public class Cat{
    .....

    @XmlElementWrapper(name="tarifes")
    @XmlElement(name="tarifa")
    public List <Tarifa> getTarifes() {...}

    .....
}
```

Generaria aquesta representació XML:

```
<catalog>
    .... elements
    <tarifes>
        <tarifa>....</tarifa>
        <tarifa>....</tarifa>
        ...
    </tarifes>
    ...
</catalog>
```

Podeu trobar més informació sobre les anotacions **JAXB** al material o en aquest enllaç: <https://howtodoinjava.com/jaxb/jaxb-annotations/>

	Codi: I71	Exercici d'avaluació contínua 1	Pàgina 17 de 17
	Versió: 03	DAM_0486B0_EAC1_Enunciat_2526S2	Lliurament: 17/03/2026